Reusing a BBB Classifier as a GRPO Reward

How the loop collapses, why, and what restrains it

S Hariharan, J Shrenik Reddy, Ansa N
{524cs0005, 124cs006, 124cs0077}@iiitk.ac.in

RECOMB 2027

Why this problem

- Whether a molecule crosses the blood–brain barrier (BBB) decides if it can act on the CNS.
- Labelled data are small: BBBP $\approx 2\,000$ compounds, B3DB $\approx 8\,000$.
- GNNs rank well on them, so it is tempting to reuse the trained network as the **reward** of a generator.
- But a generator queries the classifier on molecules *chosen to maximize its output*, so any weakness is actively sought out.

Question: what happens when we do exactly that, and which interventions actually help?

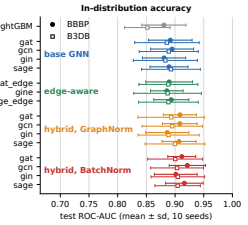
System and protocol

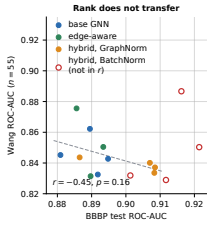
SMILES $\rightarrow$ RDKit graph $\rightarrow$ GNN $\rightarrow$ P(BBB permeable) $\Longrightarrow$ frozen reward for a GRPO generator

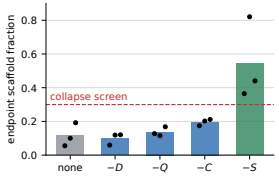
- **Leak-corrected scaffold split:** the standard Murcko grouping silently excluded acyclic molecules from every test fold; we fix this.
- **Ten seeds**; each seed is a different scaffold split, so $\pm$ includes split variance.
- **External holdouts** (Adenot, Wang) filtered to be scaffold-disjoint from both training sets.
- Frozen classifier means `eval()`: BatchNorm in train mode would make a molecule's reward depend on its group.

Classifier side: what is actually resolved

- Descriptor fusion: **+0.0099** ROC-AUC
- Batch normalization: **+0.0074**
- Jointly: **+0.0195**
- Choice of message-passing operator: **not resolved**
- In-distribution rank does not predict external rank: $r = -0.45$, $p = 0.16$, 11 models (underpowered)







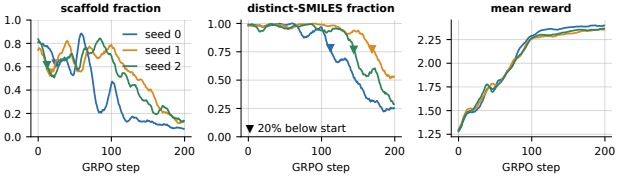
Generator and the two-part advantage

- GRPO replaces the value function by group statistics; here rewards are *zero-inflated* (invalid $\rightarrow$ flat penalty).
- Pooling zeros and valid molecules lets a few invalids set the scale for ranking the valid ones.
- **Hurdle advantage:** standardize within the valid subgroup; invalids get a fixed floor.

invalid in group of 64	0	1	2	3
spread of valid signal, pooled	1.00	0.70	0.60	0.49
spread of valid signal, hurdle	1.00	1.00	1.00	1.00

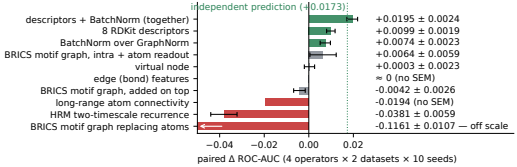
The unconstrained loop collapses — quietly

- Scaffold fraction: 0.72–0.80 $\rightarrow$ 0.07–0.14 in 200 steps
- Validity rises to 100%
- Every metric the loop originally logged looks healthy



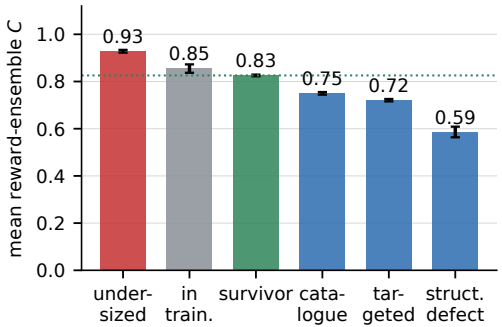
Why: what a group-standardized update cannot see

- The standardized advantage is **blind to the absolute reward level**.
- Reweighting reward terms *inside* it does not restrain the collapse.
- Only quantities acting *outside* it do: a size gate, a fixed invalidity floor, a KL penalty.



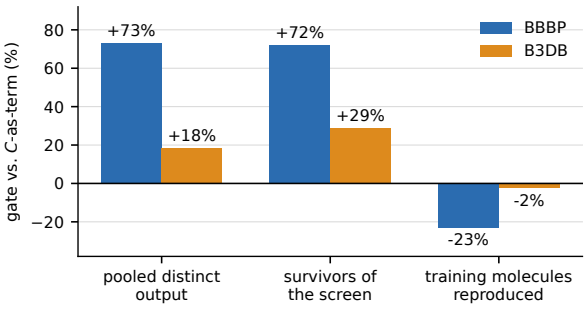
Which anchors work — including our own retractions

- We tested anchors against the unconstrained baseline and report what held up.
- Two earlier claims of ours are retracted in the paper.
- The final generator assembles BRICS fragments.



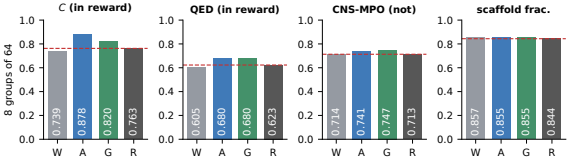
Permeability as a gate, not a direction

- BBBP: **+73%** distinct output
- Corpus reproduction: 88.9% → 68.4%
- B3DB replication: gain shrinks to **+18%**
- Mechanism: the benefit scales with how badly the policy had memorized its corpus



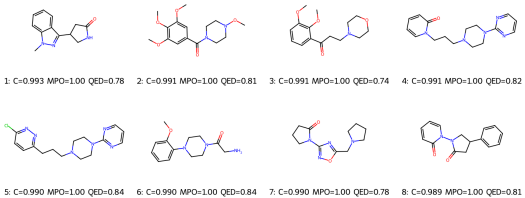
Against baselines, and where outputs stand

- In-reward terms improve: classifier $+0.126$, QED $+0.054$
- Independent CNS-MPO does not: $+0.008$, $p = 0.45$
- 72.0% of generated molecules are absent from PubChem and the training corpus



Shortlist and limitations

- The BBBP label measures **transport, not drug suitability**: a permeability objective rewards small lipophilic solvents.
- This limitation is unresolved.
- All outputs are **hypotheses, not synthesis candidates**.



Take-aways and code

1. A corrected split and ten seeds show most architecture differences are noise; descriptors and BatchNorm are what help.
2. Reusing a classifier as a reward collapses diversity while standard metrics look fine.
3. A group-standardized update is blind to absolute reward level: constrain from outside it.
4. A permeability *gate* helps most when the policy memorizes its corpus.

Code, results and run histories: `github.com/JSR996/bbb-gnn-permeability` (branch `main`, tag `recomb2027-final`); reproduce with `python -m src.pipeline`.